

MERN Stack - Top 10 High-Probability Exam Questions

Semester VI - Full-Stack Web Development

Question 1: Explain the MERN Stack Architecture with Diagram and Real-Time Example

Answer:

What is MERN Stack? MERN is a full-stack JavaScript framework consisting of: - **M** = MongoDB (NoSQL Database) - **E** = Express.js (Backend Framework) - **R** = React.js (Frontend Library) - **N** = Node.js (Runtime Environment)

MERN Architecture Flow:

```
Frontend Layer (Browser)
React.js (Components, State, Props, Hooks)
  UI Components
  HTTP Requests (fetch/axios)
  State Management
    ↓ (JSON Request)

Backend Layer (Server)
Node.js + Express.js
  HTTP Server (Port 3000/5000)
  Routing (GET, POST, PUT, DELETE)
  Middleware (CORS, Body Parser, Auth)
  Business Logic
  API Endpoints (/api/users, /api/posts, etc.)
    ↓ (Query/Insert/Update)

Database Layer
MongoDB (Cloud/Local)
  Collections (users, posts, products)
  Documents (JSON format)
  Indexes & Relationships
```

Real-Time Example: College Student Management System

Scenario: A student wants to view their exam results on a college website.

Flow: 1. **Frontend (React)** - Student clicks “View Results” button - React component sends: GET /api/results/101 - useEffect hook displays results on page

2. Backend (Express)

```
app.get('/api/results/:id', async (req, res) => {
  try {
    const studentId = req.params.id;
    const results = await Result.findById(studentId);
    res.json(results);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

3. Database (MongoDB)

- Stores: { studentId: 101, marks: 85, subject: “DSA” }
- Returns document to Express
- Express sends JSON response to React
- React displays: “Your Score: 85/100”

Advantages of MERN Stack:

- One language (JavaScript) for entire app
- Open-source and free
- High performance
- Large community support
- Easy to scale for startups
- Flexible database (NoSQL)

Question 2: What is Middleware in Express.js? Explain with Types and Real-Time Example

Answer:

What is Middleware? Middleware is a function that executes between the request and response. It has access to request (req) and response (res) objects.

Middleware Flow Diagram:

Client Request
↓

```
Middleware 1 (Logger) → next()
↓
Middleware 2 (Auth) → next()
↓
Middleware 3 (Parser) → next()
↓
Route Handler (Business Logic)
↓
Send Response to Client
```

5 Types of Middleware:

1. Application-Level Middleware

```
app.use((req, res, next) => {
  console.log("Application-level middleware");
  next();
});
```

- Applied to all routes
- Example: Logging requests

2. Router-Level Middleware

```
const router = express.Router();
router.use((req, res, next) => {
  console.log("Router-level middleware");
  next();
});
app.use('/api', router);
```

- Applied to specific routes
- Example: Admin routes only

3. Built-in Middleware

```
app.use(express.json());           // Parse JSON body
app.use(express.urlencoded());     // Parse form data
app.use(express.static('public')); // Serve static files
```

4. Third-Party Middleware

```
const cors = require('cors');
const morgan = require('morgan');

app.use(cors());           // Enable CORS
app.use(morgan('dev'));    // Log HTTP requests
```

5. Error Handling Middleware

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: "Something went wrong" });
});
```

Real-Time Example: E-Commerce Authentication

Scenario: Protecting “/dashboard” route - only logged-in users can access.

```
// Middleware to check authentication
function isAuthenticated(req, res, next) {
  const token = req.headers['authorization'];

  if (!token) {
    return res.status(401).json({ error: "Login required" });
  }

  // Verify token (simplified)
  if (token === "valid_token_123") {
    req.user = { id: 1, name: "Rahul" };
    next(); // Allow access
  } else {
    res.status(403).json({ error: "Invalid token" });
  }
}

// Apply middleware to protected route
app.get('/dashboard', isAuthenticated, (req, res) => {
  res.json({ message: `Welcome ${req.user.name}` });
});
```

Flow: - User: GET /dashboard with token - Middleware checks token - If valid → next() → Route executes - If invalid → 403 error response

Importance of next():

```
app.use((req, res, next) => {
  console.log("Request received");
  next(); // Mandatory - passes control to next middleware
});
```

// Without next() - request hangs forever

Order of Middleware Matters:

```
app.use(express.json()); // 1st: Parse body
app.use(isAuthenticated); // 2nd: Check auth
app.get('/data', (req, res)=> // 3rd: Route handler
```

```
res.json(req.user)
);
```

Question 3: Explain RESTful API Design with HTTP Methods. Design a Student Management REST API

Answer:

What is REST API? REST (Representational State Transfer) is an architectural style using HTTP to Create, Read, Update, Delete (CRUD) resources.

Core REST Principles:

Principle	Meaning
Client-Server	Browser (client) Server
Stateless	Each request has all needed info
Uniform Interface	Consistent URLs & methods
Resource-Based	Everything is a resource

HTTP Methods in REST:

Method	Operation	Purpose	Idempotent
GET	Read	Fetch data	Yes
POST	Create	Add new data	No
PUT	Update	Replace entire document	Yes
DELETE	Delete	Remove data	Yes
PATCH	Partial Update	Update specific fields	Sometimes

Student Management REST API Design:

```
const express = require('express');
const app = express();
app.use(express.json());

// In-Memory Database (for example)
let students = [
  { id: 1, name: "Rahul", dept: "IT", gpa: 3.8 },
  { id: 2, name: "Anu", dept: "CS", gpa: 3.9 }
];

// ===== GET - READ Operations =====
```

```

// 1. GET all students
app.get('/api/students', (req, res) => {
  res.json({
    success: true,
    count: students.length,
    data: students
  });
});

// 2. GET student by ID
app.get('/api/students/:id', (req, res) => {
  const id = req.params.id;
  const student = students.find(s => s.id == id);

  if (!student) {
    return res.status(404).json({
      error: "Student not found"
    });
  }

  res.json(student);
});

// 3. GET students filtered by department
app.get('/api/students', (req, res) => {
  const dept = req.query.dept;

  if (!dept) {
    return res.json(students);
  }

  const filtered = students.filter(s => s.dept === dept);
  res.json(filtered);
});

// ===== POST - CREATE Operation =====

app.post('/api/students', (req, res) => {
  const { name, dept, gpa } = req.body;

  // Validation
  if (!name || !dept || !gpa) {
    return res.status(400).json({
      error: "Name, dept, gpa required"
    });
  }
}

```

```

const newStudent = {
  id: students.length + 1,
  name,
  dept,
  gpa
};

students.push(newStudent);

res.status(201).json({
  message: "Student created successfully",
  data: newStudent
});
});

// ===== PUT - UPDATE Entire Document =====

app.put('/api/students/:id', (req, res) => {
  const id = req.params.id;
  const { name, dept, gpa } = req.body;

  const index = students.findIndex(s => s.id == id);

  if (index === -1) {
    return res.status(404).json({ error: "Student not found" });
  }

  // Replace entire document
  students[index] = { id, name, dept, gpa };

  res.json({
    message: "Student updated",
    data: students[index]
  });
});

// ===== PATCH - Partial Update =====

app.patch('/api/students/:id', (req, res) => {
  const id = req.params.id;
  const student = students.find(s => s.id == id);

  if (!student) {
    return res.status(404).json({ error: "Student not found" });
  }
}

```

```

    // Update only provided fields
    if (req.body.name) student.name = req.body.name;
    if (req.body.gpa) student.gpa = req.body.gpa;

    res.json({
      message: "Student updated",
      data: student
    });
  });

  // ===== DELETE - Remove Document =====

  app.delete('/api/students/:id', (req, res) => {
    const id = req.params.id;
    const index = students.findIndex(s => s.id == id);

    if (index === -1) {
      return res.status(404).json({ error: "Student not found" });
    }

    const deleted = students.splice(index, 1);

    res.json({
      message: "Student deleted",
      data: deleted[0]
    });
  });

  app.listen(3000, () => {
    console.log("Server running on port 3000");
  });

```

API Usage Examples:

1. GET All Students
curl http://localhost:3000/api/students
2. GET Student by ID
curl http://localhost:3000/api/students/1
3. GET Students by Department
curl "http://localhost:3000/api/students?dept=IT"
4. CREATE New Student
curl -X POST http://localhost:3000/api/students \


```
-H "Content-Type: application/json" \
-d '{"name":"Diya","dept":"IT","gpa":3.7}'
```

5. UPDATE Student (Full)

```
curl -X PUT http://localhost:3000/api/students/1 \
-H "Content-Type: application/json" \
-d '{"name":"Rahul Singh","dept":"CS","gpa":3.9}'
```

6. DELETE Student

```
curl -X DELETE http://localhost:3000/api/students/1
```

HTTP Status Codes Used:

Code	Meaning	When to Use
200	OK	GET successful
201	Created	POST successful
400	Bad Request	Invalid input
404	Not Found	Resource doesn't exist
500	Server Error	Server error

Question 4: Explain MongoDB Aggregation Pipeline with Real Example. When Would You Use It?

Answer:

What is Aggregation Pipeline? Aggregation is a MongoDB feature for advanced data processing and analysis. It works like a pipeline where documents pass through multiple stages, getting transformed at each stage.

Aggregation Pipeline Stages:

Stage	Purpose	Example
\$match	Filter documents	Find students with marks > 70
\$group	Group documents	Calculate average by department
\$sort	Sort results	Sort by marks (ascending/descending)
\$project	Select fields	Show only name and marks
\$sum	Calculate total	Total marks of all students
\$avg	Calculate average	Average GPA per department
\$count	Count documents	Count students
\$limit	Limit results	Get first 10 records
\$skip	Skip documents	Skip first 5 records

Real Database Example:

Collection: marks

```
[
  { _id: 1, name: "Rahul", subject: "DSA", marks: 85, dept: "IT" },
  { _id: 2, name: "Anu", subject: "DSA", marks: 92, dept: "IT" },
  { _id: 3, name: "John", subject: "DSA", marks: 78, dept: "CS" },
  { _id: 4, name: "Priya", subject: "DBMS", marks: 88, dept: "IT" },
  { _id: 5, name: "Vikram", subject: "DBMS", marks: 82, dept: "CS" }
]
```

Aggregation Example 1: Average Marks per Department

Query:

```
db.marks.aggregate([
  {
    $group: {
      _id: "$dept",           // Group by department
      avgMarks: { $avg: "$marks" },
      totalStudents: { $sum: 1 }
    }
  }
])
```

Output:

```
[
  { _id: "IT", avgMarks: 88.33, totalStudents: 3 },
  { _id: "CS", avgMarks: 80, totalStudents: 2 }
]
```

Aggregation Example 2: Filter + Sort + Project

Scenario: Find students with marks > 80, sorted by marks (descending), show only name and marks

```
db.marks.aggregate([
  {
    $match: { marks: { $gt: 80 } } // Filter marks > 80
  },
  {
    $sort: { marks: -1 } // Sort descending (-1), ascending (1)
  },
  {
    $project: {
      _id: 0,
      name: 1,
    }
  }
])
```

```

        marks: 1
      }
    }
  })

```

Output:

```

[
  { name: "Anu", marks: 92 },
  { name: "Priya", marks: 88 },
  { name: "Rahul", marks: 85 }
]

```

Aggregation Example 3: Count Students per Subject

```

db.marks.aggregate([
  {
    $group: {
      _id: "$subject",
      count: { $sum: 1 },
      avgMarks: { $avg: "$marks" }
    }
  },
  {
    $sort: { count: -1 }
  }
])

```

Output:

```

[
  { _id: "DSA", count: 3, avgMarks: 85 },
  { _id: "DBMS", count: 2, avgMarks: 85 }
]

```

Mongoose Aggregation Example:

```

const mongoose = require('mongoose');

// Schema
const markSchema = new mongoose.Schema({
  name: String,
  subject: String,
  marks: Number,
  dept: String
});

const Mark = mongoose.model('Mark', markSchema);

```

```

// Aggregation query
async function getStatistics() {
  try {
    const result = await Mark.aggregate([
      {
        $group: {
          _id: "$dept",
          avgMarks: { $avg: "$marks" },
          minMarks: { $min: "$marks" },
          maxMarks: { $max: "$marks" },
          count: { $sum: 1 }
        }
      },
      {
        $sort: { avgMarks: -1 }
      }
    ]);

    console.log(result);
  } catch (error) {
    console.error(error);
  }
}

getStatistics();

```

When to Use Aggregation Pipeline?

Use Case	Example
Data Analysis	Calculate average, total, min, max
Business Intelligence	Department-wise performance
Reports	Monthly sales summary
Statistics	Student performance trends
Complex Queries	Multi-stage transformations

Aggregation vs. find():

Aspect	find()	Aggregation
Complexity	Simple	Advanced
Processing	Single stage	Multi-stage pipeline
Use Case	Basic queries	Complex analysis
Performance	Fast	Slightly slower but flexible

Question 5: Explain React Hooks (useState and useEffect) with Practical Examples. Why Are They Important?

Answer:

What are React Hooks? Hooks are special functions that let you “hook into” React features in functional components. They start with the word “use”.

Why Hooks Are Important:

- Allow functional components to use state & lifecycle features
 - Code reuse is easier
 - Simpler than class components
 - More readable code
 - Industry standard (2025)
-

Hook 1: useState - Managing Component State

What is State? State is data that changes over time and causes component re-render when updated.

Syntax:

```
const [state, setState] = useState(initialValue);
```

Example 1: Simple Counter

```
import { useState } from 'react';

function Counter() {
  // state: current value
  // setState: function to update state
  // 0: initial value
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Count: {count}</h2>

      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>

      <button onClick={() => setCount(count - 1)}>
```

```

        Decrement
      </button>

      <button onClick={() => setCount(0)}>
        Reset
      </button>
    </div>
  );
}

```

```
export default Counter;
```

Flow: 1. Initial render: count = 0 2. User clicks “Increment” 3. setCount(1) called 4. Component re-renders with count = 1 5. UI updates automatically

Example 2: Toggle Login Status

```

import { useState } from 'react';

function Login() {
  const [isLoggedIn, setIsLoggedIn] = useState(false);
  const [username, setUsername] = useState('');

  return (
    <div>
      <div>
        {isLoggedIn ? (
          <div>
            <h2>Welcome, {username}!</h2>
            <button onClick={() => setIsLoggedIn(false)}>
              Logout
            </button>
          </div>
        ) : (
          <div>
            <input
              placeholder="Username"
              value={username}
              onChange={(e) => setUsername(e.target.value)}
            />
            <button onClick={() => setIsLoggedIn(true)}>
              Login
            </button>
          </div>
        )}
      </div>
    </div>
  );
}

```

```
}
```

```
export default Login;
```

Example 3: Form with Multiple States

```
import { useState } from 'react';
```

```
function StudentForm() {  
  const [formData, setFormData] = useState({  
    name: '',  
    email: '',  
    course: 'IT'  
  });  
  
  const handleChange = (e) => {  
    const { name, value } = e.target;  
    setFormData({  
      ...formData,  
      [name]: value  
    });  
  };  
  
  const handleSubmit = (e) => {  
    e.preventDefault();  
    console.log("Form Submitted:", formData);  
  };  
  
  return (  
    <form onSubmit={handleSubmit}>  
      <input  
        name="name"  
        placeholder="Name"  
        value={formData.name}  
        onChange={handleChange}  
      />  
      <input  
        name="email"  
        placeholder="Email"  
        value={formData.email}  
        onChange={handleChange}  
      />  
      <select name="course" value={formData.course} onChange={handleChange}>  
        <option>IT</option>  
        <option>CS</option>  
        <option>CE</option>  
      </select>  
    </form>  
  );  
}
```

```

        </select>
        <button type="submit">Submit</button>
      </form>
    );
  }

  export default StudentForm;

```

Hook 2: useEffect - Side Effects in Functional Components

What is useEffect? useEffect lets you perform side effects in functional components like: - Fetching data from API - Updating title - Starting timers - Setting up subscriptions

Syntax:

```

useEffect(() => {
  // Side effect code here

  return () => {
    // Cleanup code (optional)
  };
}, [dependencies]); // Dependency array

```

Example 1: Fetch Data on Component Load

```

import { useState, useEffect } from 'react';

function StudentList() {
  const [students, setStudents] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // Fetch data when component mounts
    fetch('http://localhost:3000/api/students')
      .then(res => res.json())
      .then(data => {
        setStudents(data);
        setLoading(false);
      })
      .catch(err => console.error(err));
  }, []); // Empty array = runs once on mount

  if (loading) return <h2>Loading...</h2>;

  return (

```



```

    <div>
      <h2>Students List</h2>
      <ul>
        {students.map(student => (
          <li key={student.id}>{student.name}</li>
        ))}
      </ul>
    </div>
  );
}

export default StudentList;

```

Example 2: Timer with Cleanup

```

import { useState, useEffect } from 'react';

function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => {
      setSeconds(s => s + 1);
    }, 1000);

    // Cleanup function - runs when component unmounts
    return () => clearInterval(interval);
  }, []); // Run once

  return <h2>Time: {seconds} seconds</h2>;
}

export default Timer;

```

Example 3: Update Title Based on State

```

import { useState, useEffect } from 'react';

function TitleUpdater() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    // Update document title whenever count changes
    document.title = `Count: ${count}`;
  }, [count]); // Run when count changes
}

```

```

    return (
      <div>
        <h2>Count: {count}</h2>
        <button onClick={() => setCount(count + 1)}>
          Increase
        </button>
      </div>
    );
  }

  export default TitleUpdater;

```

Dependency Array Rules:

Dependency Array	When It Runs	Use Case
[]	Once (on mount)	Fetch initial data
[count]	When 'count' changes	Re-fetch when filter changes
No array	Every render	Not recommended - causes loops

Example: Dependency Array Comparison

```

// 1. Run once on mount
useEffect(() => {
  console.log("Component mounted");
}, []);

// 2. Run when 'userId' changes
useEffect(() => {
  fetchUserData(userId);
}, [userId]);

// 3. Run every time component renders (AVOID!)
useEffect(() => {
  console.log("Rendered");
}); // No dependency array

```

Real-World Example: User Profile Component

```

import { useState, useEffect } from 'react';

function UserProfile({ userId }) {
  const [user, setUser] = useState(null);

```

```

const [posts, setPosts] = useState([]);
const [loading, setLoading] = useState(true);

useEffect(() => {
  // Fetch user data when userId changes
  setLoading(true);

  Promise.all([
    fetch(`/api/users/${userId}`).then(r => r.json()),
    fetch(`/api/posts?userId=${userId}`).then(r => r.json())
  ])
  .then(([userData, postsData]) => {
    setUser(userData);
    setPosts(postsData);
    setLoading(false);
  });

  // Update page title
  document.title = `User ${userId}`;

}, [userId]); // Re-run when userId changes

if (loading) return <p>Loading...</p>;
if (!user) return <p>User not found</p>;

return (
  <div>
    <h2>{user.name}</h2>
    <p>Email: {user.email}</p>

    <h3>Posts</h3>
    <ul>
      {posts.map(post => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  </div>
);
}

export default UserProfile;

```

Question 6: Explain JWT Authentication Flow. How Do You Protect Routes in Express? Code Example

Answer:

What is JWT (JSON Web Token)? JWT is a secure, self-contained token used for authentication and authorization.

JWT Structure:

Header.Payload.Signature

Example:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.  
eyJpZCI6MSwiZWlhaWwiOiJyYWAbGdtYWlsLmNvbSJ9.  
abcd1234efgh5678...
```

JWT Authentication Flow:

1. User Login

Username & Password sent to Server

2. Server Verification

Check credentials in database

3. Token Generation

Server creates JWT token
Includes: user id, role, expiry

4. Token Sent to Client

Client stores token (localStorage/sessionStorage)

5. Subsequent Requests

Client sends token in Authorization header
Server verifies token
If valid → Grant access
If invalid → 401 Unauthorized

Complete Implementation Example:

Step 1: Install JWT Library

```
npm install jsonwebtoken
npm install bcryptjs // For password hashing
```

Step 2: Backend - Express Server

```
const express = require('express');
const jwt = require('jsonwebtoken');
const app = express();
app.use(express.json());

const SECRET_KEY = process.env.SECRET_KEY || 'your_secret_key';

// Simulated user database
const users = [
  { id: 1, email: 'rahul@gmail.com', password: 'password123', role: 'student' },
  { id: 2, email: 'admin@college.com', password: 'admin123', role: 'admin' }
];

// ===== LOGIN ROUTE =====

app.post('/api/auth/login', (req, res) => {
  const { email, password } = req.body;

  // Find user
  const user = users.find(u => u.email === email);

  if (!user) {
    return res.status(401).json({ error: "User not found" });
  }

  // Check password (simplified - use bcrypt in production)
  if (user.password !== password) {
    return res.status(401).json({ error: "Invalid password" });
  }

  // Generate JWT Token
  const token = jwt.sign(
    { id: user.id, email: user.email, role: user.role },
    SECRET_KEY,
    { expiresIn: '24h' } // Token expires in 24 hours
  );

  res.json({
    message: "Login successful",
    token: token,
    user: { id: user.id, email: user.email, role: user.role }
  });
});
```

```

});

// ===== VERIFY TOKEN MIDDLEWARE =====

function verifyToken(req, res, next) {
  // Get token from header
  const token = req.headers['authorization']?.split(' ')[1];

  if (!token) {
    return res.status(403).json({ error: "Token required" });
  }

  jwt.verify(token, SECRET_KEY, (err, decoded) => {
    if (err) {
      return res.status(401).json({ error: "Invalid or expired token" });
    }

    // Store user info in request
    req.user = decoded;
    next();
  });
}

// ===== CHECK ROLE MIDDLEWARE =====

function checkRole(requiredRole) {
  return (req, res, next) => {
    if (req.user.role !== requiredRole) {
      return res.status(403).json({ error: "Admin access required" });
    }
    next();
  };
}

// ===== PROTECTED ROUTES =====

// 1. Student Dashboard (any logged-in user)
app.get('/api/dashboard', verifyToken, (req, res) => {
  res.json({
    message: `Welcome ${req.user.email}`,
    data: { role: req.user.role }
  });
});

// 2. Student Results (student or admin)
app.get('/api/results/:studentId', verifyToken, (req, res) => {

```

```

    if (req.user.id !== parseInt(req.params.studentId) && req.user.role !== 'admin') {
      return res.status(403).json({ error: "Unauthorized" });
    }

    res.json({
      studentId: req.params.studentId,
      results: { DSA: 85, DBMS: 92 }
    });
  });

  // 3. Admin Panel (admin only)
  app.get('/api/admin/users',
    verifyToken,
    checkRole('admin'),
    (req, res) => {
      res.json({
        message: "Admin Panel",
        users: users
      });
    }
  );

  // 4. Delete User (admin only)
  app.delete('/api/admin/users/:id',
    verifyToken,
    checkRole('admin'),
    (req, res) => {
      const userId = req.params.id;
      const index = users.findIndex(u => u.id === parseInt(userId));

      if (index === -1) {
        return res.status(404).json({ error: "User not found" });
      }

      const deleted = users.splice(index, 1);
      res.json({
        message: "User deleted",
        deleted: deleted[0]
      });
    }
  );

  app.listen(3000, () => {
    console.log("Server running on port 3000");
  });

```

Step 3: Frontend - React Implementation

```
import { useState } from 'react';

function LoginPage() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [token, setToken] = useState(localStorage.getItem('token'));

  const handleLogin = async (e) => {
    e.preventDefault();

    const response = await fetch('http://localhost:3000/api/auth/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, password })
    });

    const data = await response.json();

    if (response.ok) {
      // Store token
      localStorage.setItem('token', data.token);
      setToken(data.token);
      alert('Login successful!');
    } else {
      alert(data.error);
    }
  };

  const fetchProtectedData = async () => {
    const response = await fetch('http://localhost:3000/api/dashboard', {
      method: 'GET',
      headers: {
        'Authorization': `Bearer ${token}`
      }
    });

    const data = await response.json();
    console.log(data);
  };

  return (
    <div>
      {!token ? (
        <form onSubmit={handleLogin}>
```



```

    <h2>Login</h2>
    <input
      type="email"
      value={email}
      onChange={(e) => setEmail(e.target.value)}
      placeholder="Email"
    />
    <input
      type="password"
      value={password}
      onChange={(e) => setPassword(e.target.value)}
      placeholder="Password"
    />
    <button type="submit">Login</button>
  </form>
) : (
  <div>
    <h2>Logged In!</h2>
    <button onClick={fetchProtectedData}>
      View Dashboard
    </button>
    <button onClick={() => {
      localStorage.removeItem('token');
      setToken(null);
    }}>
      Logout
    </button>
  </div>
)}
</div>
);
}

export default LoginPage;

```

JWT Security Best Practices:

Practice	Why
Use HTTPS	Encrypt token transmission
Store in secure cookie	Prevent XSS attacks
Set expiration	Reduce token abuse risk
Use strong secret key	Harder to forge tokens
Verify signature	Detect tampered tokens

Question 7: Explain Error Handling in Express APIs. Design Error Handling Middleware

Answer:

Why Error Handling is Important: - Prevents application crashes - Provides clear error messages to clients - Improves debugging - Enhances user experience - Improves security (don't leak internal details)

Types of Errors in APIs:

Error Type	Status Code	Cause
Bad Request	400	Invalid input
Unauthorized	401	Missing/invalid token
Forbidden	403	Insufficient permissions
Not Found	404	Resource doesn't exist
Server Error	500	Internal error
Conflict	409	Data duplication

Error Handling Patterns:

Pattern 1: Basic Try-Catch in Routes

```
app.get('/api/students/:id', async (req, res) => {
  try {
    const id = req.params.id;
    const student = await Student.findById(id);

    if (!student) {
      return res.status(404).json({
        error: "Student not found"
      });
    }

    res.json(student);

  } catch (error) {
    res.status(500).json({
      error: "Internal server error",
      message: error.message
    });
  }
});
```

Pattern 2: Custom Error Class

```

class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
  }
}

app.get('/api/students/:id', async (req, res, next) => {
  try {
    const student = await Student.findById(req.params.id);

    if (!student) {
      throw new AppError("Student not found", 404);
    }

    res.json(student);

  } catch (error) {
    next(error); // Pass to error middleware
  }
});

```

Pattern 3: Centralized Error Handling Middleware

```

// Error handler middleware (must be last)
app.use((err, req, res, next) => {
  const statusCode = err.statusCode || 500;
  const message = err.message || "Internal Server Error";

  res.status(statusCode).json({
    success: false,
    error: {
      statusCode: statusCode,
      message: message
    }
  });
});

```

Complete Express Server with Comprehensive Error Handling:

```

const express = require('express');
const mongoose = require('mongoose');
const app = express();

app.use(express.json());

// ===== CUSTOM ERROR CLASS =====

```

```

class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
    this.isOperational = true;
  }
}

// ===== MONGODB SCHEMA =====

const studentSchema = new mongoose.Schema({
  rollNo: {
    type: Number,
    required: true,
    unique: true
  },
  name: {
    type: String,
    required: true
  },
  email: {
    type: String,
    required: true,
    unique: true
  },
  dept: String,
  marks: Number
});

const Student = mongoose.model('Student', studentSchema);

// ===== API ROUTES WITH ERROR HANDLING =====

// 1. GET all students
app.get('/api/students', async (req, res, next) => {
  try {
    const students = await Student.find();

    if (students.length === 0) {
      throw new AppError("No students found", 404);
    }

    res.json({
      success: true,
      count: students.length,

```

```

        data: students
    });

    } catch (error) {
        next(error);
    }
});

// 2. GET student by ID
app.get('/api/students/:id', async (req, res, next) => {
    try {
        const { id } = req.params;

        // Validate ID format
        if (!mongoose.Types.ObjectId.isValid(id)) {
            throw new AppError("Invalid student ID format", 400);
        }

        const student = await Student.findById(id);

        if (!student) {
            throw new AppError("Student not found", 404);
        }

        res.json({
            success: true,
            data: student
        });

    } catch (error) {
        next(error);
    }
});

// 3. CREATE student
app.post('/api/students', async (req, res, next) => {
    try {
        const { rollNo, name, email, dept } = req.body;

        // Validation
        if (!rollNo || !name || !email) {
            throw new AppError("Roll No, Name, and Email are required", 400);
        }

        // Check if student already exists
        const exists = await Student.findOne({ rollNo });

```

```

    if (exists) {
      throw new AppError("Student with this roll number already exists", 409);
    }

    const student = new Student({ rollNo, name, email, dept });
    await student.save();

    res.status(201).json({
      success: true,
      message: "Student created successfully",
      data: student
    });

  } catch (error) {
    next(error);
  }
});

// 4. UPDATE student
app.put('/api/students/:id', async (req, res, next) => {
  try {
    const { id } = req.params;
    const { name, email, dept } = req.body;

    if (!mongoose.Types.ObjectId.isValid(id)) {
      throw new AppError("Invalid student ID", 400);
    }

    const student = await Student.findByIdAndUpdate(
      id,
      { name, email, dept },
      { new: true, runValidators: true }
    );

    if (!student) {
      throw new AppError("Student not found", 404);
    }

    res.json({
      success: true,
      message: "Student updated successfully",
      data: student
    });

  } catch (error) {
    next(error);
  }
});

```

```

    }
  });

  // 5. DELETE student
  app.delete('/api/students/:id', async (req, res, next) => {
    try {
      const { id } = req.params;

      if (!mongoose.Types.ObjectId.isValid(id)) {
        throw new AppError("Invalid student ID", 400);
      }

      const student = await Student.findByIdAndDelete(id);

      if (!student) {
        throw new AppError("Student not found", 404);
      }

      res.json({
        success: true,
        message: "Student deleted successfully",
        data: student
      });

    } catch (error) {
      next(error);
    }
  });

  // ===== INVALID ROUTE HANDLER =====

  app.use('*', (req, res, next) => {
    throw new AppError(`Cannot find ${req.originalUrl} on this server`, 404);
  });

  // ===== GLOBAL ERROR HANDLING MIDDLEWARE =====
  // This must be the LAST middleware

  app.use((err, req, res, next) => {

    // Mongoose validation error
    if (err.name === 'ValidationError') {
      const messages = Object.values(err.errors).map(e => e.message);
      return res.status(400).json({
        success: false,
        error: {

```

```

        type: "Validation Error",
        messages: messages
      }
    });
  }

  // Mongoose duplicate key error
  if (err.code === 11000) {
    const field = Object.keys(err.keyPattern)[0];
    return res.status(409).json({
      success: false,
      error: `${field} already exists`
    });
  }

  // JWT errors
  if (err.name === 'JsonWebTokenError') {
    return res.status(401).json({
      success: false,
      error: "Invalid token"
    });
  }

  // Custom AppError
  if (err.isOperational) {
    return res.status(err.statusCode).json({
      success: false,
      error: err.message
    });
  }

  // Unknown error
  console.error("UNKNOWN ERROR:", err);

  res.status(err.statusCode || 500).json({
    success: false,
    error: {
      statusCode: err.statusCode || 500,
      message: err.message || "Internal Server Error"
    }
  });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});

```



```
});
```

Best Practices for Error Handling:

Practice	Example
Use meaningful messages	“Error” vs “Email already registered”
Use correct status codes	400 for client, 500 for server
Don’t expose internals	Stack traces vs Generic message
Validate input	Check types, required fields
Log errors	Save for debugging
Use centralized handler	One place for all errors

Question 8: Compare SQL vs NoSQL. Why Is MongoDB Chosen for MERN Stack?

Answer:

SQL vs NoSQL Comparison:

Aspect	SQL (Relational)	NoSQL (Document)
Data Structure	Tables & Rows	Documents (JSON)
Schema	Fixed schema required	Flexible, dynamic schema
Relationships	JOIN operations	Embedded or referenced
Scalability	Vertical	Horizontal (easy)
Examples	MySQL, PostgreSQL	MongoDB, Cassandra
Use Case	Structured data	Unstructured/semi-structured
Speed	Slower (complex JOINS)	Faster (denormalized)
ACID	Full support	Some support
Learning Curve	Moderate	Easy for developers

Visual Comparison:

SQL (Relational Database)

Students Table:

ID	Name	Dept
1	Rahul	IT
2	Anu	CS

Marks Table:

ID	StudentID	Marks
1	1	85
2	2	90

Query (with JOIN):

```
SELECT s.name, m.marks
FROM students s
JOIN marks m ON s.id = m.studentID;
```

NoSQL (MongoDB)

Students Collection:

```
{
  "_id": ObjectId("1"),
  "name": "Rahul",
  "dept": "IT",
  "marks": [
    { "subject": "DSA", "marks": 85 },
    { "subject": "DBMS", "marks": 88 }
  ]
}

{
  "_id": ObjectId("2"),
  "name": "Anu",
  "dept": "CS",
  "marks": [
    { "subject": "DSA", "marks": 90 }
  ]
}
```

Query (no JOIN needed):

```
db.students.find({ "marks.marks": { $gt: 85 } })
```

Why MongoDB for MERN Stack?

1. JavaScript Integration - MongoDB uses JSON format - React already uses JSON - Express handles JSON naturally - Node.js has excellent MongoDB libraries

2. Flexible Schema

// Same collection can have different structures

```

{
  name: "Student 1",
  email: "stu1@college.com",
  age: 20,
  phone: "9876543210"
}

{
  name: "Student 2",
  email: "stu2@college.com",
  // No age field needed
  // Can add new fields anytime
}

```

3. Easy to Scale

Single Server:

Database → Replica Set (3 servers) → Sharded Cluster

No complex migration needed!

4. Rapid Development - Schema not required upfront - Changes don't need database migrations - Quicker development cycle

5. Mongoose for Structure - Even though flexible, we can enforce schema - Validation & relationships - Best of both worlds

Real-World Example: College System

```

// Student Document
{
  _id: ObjectId("..."),
  name: "Rahul",
  email: "rahul@college.com",
  dept: "IT",
  year: "TY",
  marks: [
    { subject: "DSA", marks: 85 },
    { subject: "DBMS", marks: 92 }
  ],
  attendance: 85,
  fees: {
    total: 50000,
    paid: 50000,
    pending: 0
  },
  activities: [
    { name: "Sports", year: 2024 },

```

```
{ name: "Coding Club", year: 2024 }
]
```

SQL Approach: - Students table - Marks table - Attendance table - Fees table
- Activities table - Need 5 JOINS to get complete data

MongoDB Approach: - Single student document - Direct access to all data -
No JOINS needed

Performance Comparison:

SQL Query:

```
SELECT s.*, m.marks, a.attendance, f.fees
FROM students s
LEFT JOIN marks m ON s.id = m.student_id
LEFT JOIN attendance a ON s.id = a.student_id
LEFT JOIN fees f ON s.id = f.student_id
WHERE s.dept = 'IT';
```

Complex, slow for large datasets

MongoDB Query:

```
db.students.find({ dept: "IT" })
```

Simple, fast, returns complete data

When NOT to Use MongoDB:

Scenario	Why	Use Instead
Complex transactions	Limited ACID	PostgreSQL
Complex reporting	Hard to aggregate	Data warehouse
Strict data integrity	Flexible schema risky	SQL

Summary:

MongoDB + MERN is ideal for: - Web & mobile apps - Real-time applications
- Rapid prototyping - Startups & MVPs - Content management

Not ideal for: - Banking systems - Complex business logic - Data warehouse

Question 9: Explain React Components, Props, and State. Design a Complete Product Listing Component

Answer:

Core Concepts:

1. **Component** = Reusable UI piece
2. **Props** = Data sent FROM parent TO child (read-only)
3. **State** = Data managed WITHIN component (changeable)

Comparison:

Aspect	State	Props
Who manages?	Component	Parent component
Can change?	Yes (setState)	No (read-only)
Re-render?	Yes	Yes
Use case	Dynamic data	Pass data down

Complete Product Listing Component Example:

```
import { useState, useEffect } from 'react';

// ===== PRODUCT CARD COMPONENT (Child) =====

// This component receives props from parent
function ProductCard({ product, onAddToCart, onDelete }) {
  const [quantity, setQuantity] = useState(1);

  return (
    <div style={{
      border: '1px solid #ddd',
      padding: '20px',
      borderRadius: '8px',
      marginBottom: '20px'
    }}>
      <h3>{product.name}</h3>
      <p>Price: {product.price}</p>
      <p>Category: {product.category}</p>
      <p>Stock: {product.stock}</p>

      <div>
        <input
          type="number"
          min="1"
          max={product.stock}
        />
      </div>
    </div>
  );
}
```

```

        value={quantity}
        onChange={(e) => setQuantity(Number(e.target.value))}
      />

      <button
        onClick={() => onAddToCart(product.id, quantity)}
        disabled={product.stock === 0}
      >
        Add to Cart
      </button>

      <button
        onClick={() => onDelete(product.id)}
        style={{ color: 'red' }}
      >
        Delete
      </button>
    </div>
  </div>
);
}

// ===== FILTER COMPONENT (Child) =====

function FilterSection({ selectedCategory, onFilterChange }) {
  const categories = ['All', 'Electronics', 'Clothing', 'Books', 'Food'];

  return (
    <div style={{ marginBottom: '20px' }}>
      <h3>Filter by Category</h3>
      {categories.map(cat => (
        <button
          key={cat}
          onClick={() => onFilterChange(cat)}
          style={{
            padding: '8px 15px',
            margin: '5px',
            backgroundColor: selectedCategory === cat ? '#007bff' : '#f0f0f0',
            color: selectedCategory === cat ? 'white' : 'black',
            border: 'none',
            borderRadius: '5px',
            cursor: 'pointer'
          }}
        >
          {cat}
        </button>
      ))}
    </div>
  );
}

```

```

    ))}
  </div>
);
}

// ===== SEARCH COMPONENT (Child) =====

function SearchBar({ searchTerm, onSearchChange }) {
  return (
    <div style={{ marginBottom: '20px' }}>
      <input
        type="text"
        placeholder="Search products..."
        value={searchTerm}
        onChange={(e) => onSearchChange(e.target.value)}
        style={{
          padding: '10px',
          width: '300px',
          borderRadius: '5px',
          border: '1px solid #ddd',
          fontSize: '16px'
        }}
      />
    </div>
  );
}

// ===== PRODUCT LISTING COMPONENT (Parent) =====

function ProductListing() {
  // State: Manage products, filters, search
  const [products, setProducts] = useState([]);
  const [filteredProducts, setFilteredProducts] = useState([]);
  const [selectedCategory, setSelectedCategory] = useState('All');
  const [searchTerm, setSearchTerm] = useState('');
  const [loading, setLoading] = useState(true);
  const [cart, setCart] = useState([]);

  // Fetch products on component mount
  useEffect(() => {
    fetchProducts();
  }, []);

  // Filter products whenever category or search changes
  useEffect(() => {
    filterProducts();
  }, [selectedCategory, searchTerm]);
}

```

```

}, [products, selectedCategory, searchTerm]);

const fetchProducts = async () => {
  try {
    // Simulated API call
    const mockProducts = [
      { id: 1, name: 'Laptop', price: 45000, category: 'Electronics', stock: 5 },
      { id: 2, name: 'Mobile', price: 25000, category: 'Electronics', stock: 10 },
      { id: 3, name: 'T-Shirt', price: 500, category: 'Clothing', stock: 20 },
      { id: 4, name: 'Jeans', price: 1500, category: 'Clothing', stock: 15 },
      { id: 5, name: 'JavaScript Book', price: 800, category: 'Books', stock: 8 },
      { id: 6, name: 'Python Book', price: 900, category: 'Books', stock: 12 },
      { id: 7, name: 'Pizza', price: 300, category: 'Food', stock: 50 },
      { id: 8, name: 'Burger', price: 250, category: 'Food', stock: 30 }
    ];

    setProducts(mockProducts);
    setLoading(false);

  } catch (error) {
    console.error('Error fetching products:', error);
    setLoading(false);
  }
};

const filterProducts = () => {
  let filtered = products;

  // Filter by category
  if (selectedCategory !== 'All') {
    filtered = filtered.filter(p => p.category === selectedCategory);
  }

  // Filter by search term
  if (searchTerm) {
    filtered = filtered.filter(p =>
      p.name.toLowerCase().includes(searchTerm.toLowerCase())
    );
  }

  setFilteredProducts(filtered);
};

const handleAddToCart = (productId, quantity) => {
  const product = products.find(p => p.id === productId);

```



```

    setCart([...cart, {
      ...product,
      orderQuantity: quantity
    }]);

    alert(`Added ${quantity} ${product.name}(s) to cart!`);
  };

  const handleDelete = (productId) => {
    setProducts(products.filter(p => p.id !== productId));
  };

  const handleFilterChange = (category) => {
    setSelectedCategory(category);
  };

  const handleSearchChange = (term) => {
    setSearchTerm(term);
  };

  if (loading) return <h2>Loading products...</h2>;

  return (
    <div style={{ padding: '20px', fontFamily: 'Arial' }}>
      <h1> Product Store</h1>

      /* Cart Summary */
      <div style={{
        backgroundColor: '#f9f9f9',
        padding: '15px',
        borderRadius: '5px',
        marginBottom: '20px'
      }}>
        <h3>Cart Items: {cart.length}</h3>
        {cart.length > 0 && (
          <button onClick={() => setCart([])}>Clear Cart</button>
        )}
      </div>

      /* Search Bar */
      <SearchBar
        searchTerm={searchTerm}
        onSearchChange={handleSearchChange}
      />

      /* Filter Buttons */

```

```

    <FilterSection
      selectedCategory={selectedCategory}
      onFilterChange={handleFilterChange}
    />

    {/* Products Count */}
    <p>
      Found <strong>{filteredProducts.length}</strong> product(s)
    </p>

    {/* Product List */}
    {filteredProducts.length === 0 ? (
      <p>No products found. Try a different search or category.</p>
    ) : (
      <div>
        {filteredProducts.map(product => (
          <ProductCard
            key={product.id}
            product={product}
            onAddToCart={handleAddToCart}
            onDelete={handleDelete}
          />
        ))}
      </div>
    )}
  </div>
);
}

export default ProductListing;

```

Component Hierarchy:

```

ProductListing (Parent - State management)
  searchBar (Child - Receives searchTerm, onSearchChange props)
  FilterSection (Child - Receives selectedCategory, onFilterChange props)
  ProductCard (Child - Receives product, onAddToCart, onDelete props)
    Input for quantity
    Add to Cart Button

```

Key Concepts Demonstrated:

Concept	Example
Props	product, onAddToCart passed from parent to child

Concept	Example
State	<code>products</code> , <code>selectedCategory</code> , <code>searchTerm</code> managed in parent
useEffect	Fetch products on mount, filter on dependency change
Conditional Rendering	Show “No products found” if list is empty
Event Handlers	<code>onClick</code> , <code>onChange</code> events
Array Methods	<code>filter()</code> , <code>map()</code> , <code>find()</code>

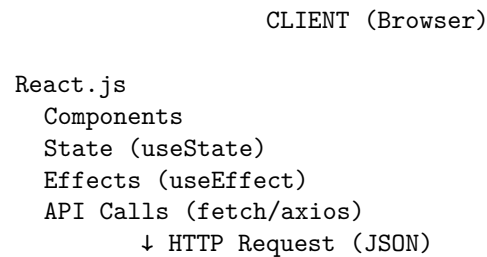
Data Flow in This Component:

1. Component Mounts
↓
2. `useEffect` runs → `fetchProducts()`
↓
3. `products` state updated → Component re-renders
↓
4. User types in search → `searchTerm` state updates
↓
5. `useEffect` triggers → `filterProducts()` runs
↓
6. `filteredProducts` updates → UI re-renders with filtered results
↓
7. User clicks "Add to Cart" → `cart` state updates → Alert shown

Question 10: Explain Full-Stack Integration: React Frontend + Express Backend + MongoDB. Complete Workflow Example

Answer:

Full-Stack Architecture:



SERVER (Node.js)

- Express.js
 - HTTP Server (Port 3000/5000)
 - Routes (GET, POST, PUT, DELETE)
 - Middleware
 - Controllers (Business Logic)
 - Authentication
 - Error Handling
 - ↓ Query/Commands

DATABASE

- MongoDB
 - Collections
 - Documents
 - Indexes
 - Schema (with Mongoose)

Complete Project: Student Grade Management System

Project Structure:

```
mern-student-app/  
  frontend/  
    src/  
      components/  
        StudentList.jsx  
        StudentForm.jsx  
        GradeCard.jsx  
      App.jsx  
      package.json  
  
  backend/  
    models/  
      Student.js  
    routes/  
      students.js  
    controllers/  
      studentController.js
```

```
server.js  
package.json
```

```
README.md
```

BACKEND CODE (Express + MongoDB)

1. Student Model (models/Student.js)

```
const mongoose = require('mongoose');  
  
const studentSchema = new mongoose.Schema({  
  name: {  
    type: String,  
    required: true  
  },  
  email: {  
    type: String,  
    required: true,  
    unique: true  
  },  
  rollNo: {  
    type: Number,  
    required: true,  
    unique: true  
  },  
  dept: {  
    type: String,  
    enum: ['IT', 'CS', 'EC', 'ME'],  
    required: true  
  },  
  grades: {  
    dsa: { type: Number, min: 0, max: 100 },  
    dbms: { type: Number, min: 0, max: 100 },  
    oop: { type: Number, min: 0, max: 100 },  
    cn: { type: Number, min: 0, max: 100 }  
  },  
  gpa: Number,  
  createdAt: {  
    type: Date,  
    default: Date.now  
  }  
});  
  
module.exports = mongoose.model('Student', studentSchema);
```

2. Student Controller (controllers/studentController.js)

```
const Student = require('../models/Student');

// GET all students
exports.getAllStudents = async (req, res) => {
  try {
    const students = await Student.find();

    if (students.length === 0) {
      return res.status(404).json({
        success: false,
        message: "No students found"
      });
    }

    res.json({
      success: true,
      count: students.length,
      data: students
    });

  } catch (error) {
    res.status(500).json({
      success: false,
      error: error.message
    });
  }
};

// GET student by ID
exports.getStudentById = async (req, res) => {
  try {
    const student = await Student.findById(req.params.id);

    if (!student) {
      return res.status(404).json({
        success: false,
        message: "Student not found"
      });
    }

    res.json({
      success: true,
      data: student
    });
  }
};
```

```

    } catch (error) {
      res.status(500).json({
        success: false,
        error: error.message
      });
    }
  };

  // CREATE student
  exports.createStudent = async (req, res) => {
    try {
      const { name, email, rollNo, dept, grades } = req.body;

      // Validation
      if (!name || !email || !rollNo || !dept) {
        return res.status(400).json({
          success: false,
          error: "All fields are required"
        });
      }

      // Check if email exists
      const exists = await Student.findOne({ email });
      if (exists) {
        return res.status(409).json({
          success: false,
          error: "Email already registered"
        });
      }

      // Calculate GPA
      const gpa = calculateGPA(grades);

      const student = new Student({
        name,
        email,
        rollNo,
        dept,
        grades,
        gpa
      });

      await student.save();

      res.status(201).json({

```

```

        success: true,
        message: "Student created successfully",
        data: student
    });

    } catch (error) {
        res.status(500).json({
            success: false,
            error: error.message
        });
    }
};

// UPDATE student grades
exports.updateGrades = async (req, res) => {
    try {
        const { id } = req.params;
        const { grades } = req.body;

        const student = await Student.findById(id);

        if (!student) {
            return res.status(404).json({
                success: false,
                error: "Student not found"
            });
        }

        student.grades = grades;
        student.gpa = calculateGPA(grades);

        await student.save();

        res.json({
            success: true,
            message: "Grades updated",
            data: student
        });

    } catch (error) {
        res.status(500).json({
            success: false,
            error: error.message
        });
    }
};

```



```

// Helper function
function calculateGPA(grades) {
  if (!grades) return 0;
  const values = Object.values(grades).filter(g => g !== undefined);
  if (values.length === 0) return 0;
  return (values.reduce((a, b) => a + b) / values.length / 10).toFixed(2);
}

// DELETE student
exports.deleteStudent = async (req, res) => {
  try {
    const student = await Student.findByIdAndDelete(req.params.id);

    if (!student) {
      return res.status(404).json({
        success: false,
        error: "Student not found"
      });
    }

    res.json({
      success: true,
      message: "Student deleted",
      data: student
    });

  } catch (error) {
    res.status(500).json({
      success: false,
      error: error.message
    });
  }
};

```

3. Routes (routes/students.js)

```

const express = require('express');
const router = express.Router();
const {
  getAllStudents,
  getStudentById,
  createStudent,
  updateGrades,
  deleteStudent
} = require('../controllers/studentController');

```

```

// Routes
router.get('/', getAllStudents);
router.get('/:id', getStudentById);
router.post('/', createStudent);
router.put('/:id/grades', updateGrades);
router.delete('/:id', deleteStudent);

module.exports = router;

4. Main Server File (server.js)

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const app = express();

// Middleware
app.use(cors());
app.use(express.json());

// MongoDB Connection
mongoose.connect(process.env.MONGO_URI || 'mongodb://localhost:27017/studentDB')
  .then(() => console.log('MongoDB Connected'))
  .catch(err => console.error('MongoDB Error:', err));

// Routes
app.use('/api/students', require('./routes/students'));

// Error handling
app.use((err, req, res, next) => {
  res.status(500).json({
    success: false,
    error: err.message
  });
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});

```

FRONTEND CODE (React)

1. Student List Component (StudentList.jsx)

```
import { useState, useEffect } from 'react';
import axios from 'axios';
import StudentForm from './StudentForm';
import GradeCard from './GradeCard';

const API_URL = 'http://localhost:5000/api/students';

function StudentList() {
  const [students, setStudents] = useState([]);
  const [loading, setLoading] = useState(true);
  const [showForm, setShowForm] = useState(false);

  // Fetch students on mount
  useEffect(() => {
    fetchStudents();
  }, []);

  const fetchStudents = async () => {
    try {
      setLoading(true);
      const response = await axios.get(API_URL);

      if (response.data.success) {
        setStudents(response.data.data);
      }
    } catch (error) {
      console.error('Error fetching students:', error);
      alert('Failed to load students');
    } finally {
      setLoading(false);
    }
  };

  const handleDeleteStudent = async (id) => {
    if (!window.confirm('Delete this student?')) return;

    try {
      const response = await axios.delete(`${API_URL}/${id}`);

      if (response.data.success) {
        setStudents(students.filter(s => s._id !== id));
      }
    }
  };
}
```

```

        alert('Student deleted successfully');
    }

    } catch (error) {
        console.error('Error deleting student:', error);
        alert('Failed to delete student');
    }
};

const handleStudentAdded = () => {
    fetchStudents();
    setShowForm(false);
};

if (loading) return <h2>Loading students...</h2>;

return (
    <div style={{ padding: '20px' }}>
        <h1> Student Grade Management</h1>

        <button
            onClick={() => setShowForm(!showForm)}
            style={{
                padding: '10px 20px',
                marginBottom: '20px',
                backgroundColor: '#007bff',
                color: 'white',
                border: 'none',
                borderRadius: '5px',
                cursor: 'pointer'
            }}
        >
            {showForm ? 'Cancel' : 'Add New Student'}
        </button>

        {showForm && <StudentForm onSuccess={handleStudentAdded} />}

        <div style={{ marginTop: '20px' }}>
            <h2>Students ({students.length})</h2>

            {students.length === 0 ? (
                <p>No students found. Add one to get started!</p>
            ) : (
                students.map(student => (
                    <div
                        key={student._id}

```

```

        style={{
          border: '1px solid #ddd',
          padding: '15px',
          marginBottom: '15px',
          borderRadius: '8px'
        }}
      >
        <h3>{student.name}</h3>
        <p>Email: {student.email}</p>
        <p>Roll No: {student.rollNo}</p>
        <p>Department: {student.dept}</p>
        <p>GPA: {student.gpa}</p>

        <GradeCard
          studentId={student._id}
          grades={student.grades}
          onGradesUpdate={fetchStudents}
        />

        <button
          onClick={() => handleDeleteStudent(student._id)}
          style={{
            padding: '8px 15px',
            backgroundColor: '#dc3545',
            color: 'white',
            border: 'none',
            borderRadius: '5px',
            cursor: 'pointer',
            marginTop: '10px'
          }}
        >
          Delete Student
        </button>
      </div>
    ))
  })
</div>
</div>
);
}

```

```
export default StudentList;
```

2. Student Form Component (StudentForm.jsx)

```
import { useState } from 'react';
import axios from 'axios';
```

```

const API_URL = 'http://localhost:5000/api/students';

function StudentForm({ onSuccess }) {
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    rollNo: '',
    dept: 'IT',
    grades: {
      dsa: '',
      dbms: '',
      oop: '',
      cn: ''
    }
  });

  const [loading, setLoading] = useState(false);

  const handleChange = (e) => {
    const { name, value } = e.target;

    if (name in formData.grades) {
      setFormData({
        ...formData,
        grades: {
          ...formData.grades,
          [name]: value ? Number(value) : ''
        }
      });
    } else {
      setFormData({
        ...formData,
        [name]: value
      });
    }
  };

  const handleSubmit = async (e) => {
    e.preventDefault();

    // Validation
    if (!formData.name || !formData.email || !formData.rollNo) {
      alert('Please fill all required fields');
      return;
    }
  };

```

```

try {
  setLoading(true);

  const response = await axios.post(API_URL, formData);

  if (response.data.success) {
    alert('Student added successfully!');
    onSuccess();
  }

} catch (error) {
  console.error('Error adding student:', error);
  alert(error.response?.data?.error || 'Failed to add student');
} finally {
  setLoading(false);
}
};

return (
  <form
    onSubmit={handleSubmit}
    style={{
      border: '1px solid #ddd',
      padding: '20px',
      borderRadius: '8px',
      maxWidth: '500px'
    }}
  >
    <h2>Add Student</h2>

    <input
      type="text"
      name="name"
      placeholder="Full Name"
      value={formData.name}
      onChange={handleChange}
      required
      style={{ width: '100%', padding: '8px', marginBottom: '10px' }}
    />

    <input
      type="email"
      name="email"
      placeholder="Email"
      value={formData.email}

```

```

        onChange={handleChange}
        required
        style={{ width: '100%', padding: '8px', marginBottom: '10px' }}
      />

      <input
        type="number"
        name="rollNo"
        placeholder="Roll Number"
        value={formData.rollNo}
        onChange={handleChange}
        required
        style={{ width: '100%', padding: '8px', marginBottom: '10px' }}
      />

      <select
        name="dept"
        value={formData.dept}
        onChange={handleChange}
        style={{ width: '100%', padding: '8px', marginBottom: '10px' }}
      >
        <option>IT</option>
        <option>CS</option>
        <option>EC</option>
        <option>ME</option>
      </select>

      <h4>Grades (out of 100)</h4>

      {[ 'dsa', 'dbms', 'oop', 'cn' ].map(subject => (
        <input
          key={subject}
          type="number"
          name={subject}
          placeholder={subject.toUpperCase()}
          value={formData.grades[subject]}
          onChange={handleChange}
          min="0"
          max="100"
          style={{ width: '100%', padding: '8px', marginBottom: '10px' }}
        />
      ))}

      <button
        type="submit"
        disabled={loading}

```



```

        style={{
          width: '100%',
          padding: '10px',
          backgroundColor: '#28a745',
          color: 'white',
          border: 'none',
          borderRadius: '5px',
          cursor: loading ? 'not-allowed' : 'pointer'
        }}
      >
        {loading ? 'Adding...' : 'Add Student'}
      </button>
    </form>
  );
}

```

```
export default StudentForm;
```

3. Grade Card Component (GradeCard.jsx)

```

import { useState } from 'react';
import axios from 'axios';

const API_URL = 'http://localhost:5000/api/students';

function GradeCard({ studentId, grades, onGradesUpdate }) {
  const [isEditing, setIsEditing] = useState(false);
  const [editedGrades, setEditedGrades] = useState(grades);

  const handleChange = (e) => {
    const { name, value } = e.target;
    setEditedGrades({
      ...editedGrades,
      [name]: value ? Number(value) : ''
    });
  };

  const handleSave = async () => {
    try {
      const response = await axios.put(`${API_URL}/${studentId}/grades`, {
        grades: editedGrades
      });

      if (response.data.success) {
        alert('Grades updated!');
        setIsEditing(false);
        onGradesUpdate();
      }
    }
  };
}

```

```

    }

    } catch (error) {
      alert('Failed to update grades');
    }
  };

  if (!isEditing) {
    return (
      <div style={{ marginTop: '10px' }}>
        <h4>Grades:</h4>
        <ul>
          <li>DSA: {grades?.dsa || 'N/A'}</li>
          <li>DBMS: {grades?.dbms || 'N/A'}</li>
          <li>OOP: {grades?.oop || 'N/A'}</li>
          <li>CN: {grades?.cn || 'N/A'}</li>
        </ul>
        <button
          onClick={() => setIsEditing(true)}
          style={{
            padding: '8px 15px',
            backgroundColor: '#17a2b8',
            color: 'white',
            border: 'none',
            borderRadius: '5px',
            cursor: 'pointer'
          }}
        >
          Edit Grades
        </button>
      </div>
    );
  }

  return (
    <div style={{ marginTop: '10px' }}>
      <h4>Edit Grades:</h4>
      {[ 'dsa', 'dbms', 'oop', 'cn' ].map(subject => (
        <input
          key={subject}
          type="number"
          name={subject}
          placeholder={subject.toUpperCase()}
          value={editedGrades[subject] || ''}
          onChange={handleChange}
          min="0"

```

```

        max="100"
        style={{ marginRight: '10px', padding: '5px' }}
      />
    ))}
    <button
      onClick={handleSave}
      style={{
        padding: '8px 15px',
        backgroundColor: '#28a745',
        color: 'white',
        border: 'none',
        borderRadius: '5px',
        cursor: 'pointer'
      }}
    >
      Save
    </button>
    <button
      onClick={() => setIsEditing(false)}
      style={{
        padding: '8px 15px',
        backgroundColor: '#dc3545',
        color: 'white',
        border: 'none',
        borderRadius: '5px',
        cursor: 'pointer',
        marginLeft: '10px'
      }}
    >
      Cancel
    </button>
  </div>
);
}

```

```
export default GradeCard;
```

4. Main App Component (App.jsx)

```

import StudentList from './components/StudentList';
import './App.css';

function App() {
  return (
    <div className="App">
      <StudentList />
    </div>
  );
}

```

```
    );  
  }  
  
  export default App;
```

Data Flow Summary:

1. USER ACTION (React)
User clicks "Add Student"
2. FORM SUBMISSION (React)
POST request to /api/students with JSON data
3. SERVER PROCESSING (Express)
Middleware parses JSON (express.json())
Route handler receives request
Controller validates data
Calls Student.create()
4. DATABASE OPERATION (MongoDB)
Mongoose validates schema
Inserts document
Returns saved document
5. RESPONSE (Express → React)
Sends back 201 + student data
6. STATE UPDATE (React)
Response parsed
students state updated
Component re-renders with new student

Installation & Running:

Backend:

```
cd backend  
npm install  
npm start
```

Frontend:

```
cd frontend  
npm create vite@latest . -- --template react  
npm install axios  
npm run dev
```

Summary Table: All 10 Topics

#	Topic	Key Concept	Real-World Use
1	MERN Stack	Full-stack JavaScript	Complete web applications
2	Middleware	Request-Response interceptor	Authentication, logging
3	REST API	CRUD operations	Mobile apps, web services
4	Aggregation	Data analysis pipeline	Reports, statistics
5	React Hooks	State & Effects in functions	Dynamic UI updates
6	JWT Auth	Secure tokens	Protected routes
7	Error Handling	Try-catch & middleware	Reliability
8	SQL vs NoSQL	Database choice	Performance optimization
9	Components/Props/State	React fundamentals	Reusable UI code
10	Full-Stack Integration	Complete workflow	Production applications

Exam Tips:

Always include examples - Real code > Theory
Draw diagrams - Data flow, architecture
Explain “why” - Not just “how”
Use real-world scenarios - Students, products, etc.
Compare alternatives - REST vs SOAP, SQL vs NoSQL
Mention best practices - Error handling, security

Good Luck with Your Exam!